



INSTITUT TEKNOLOGI DEL

BASIC JAVA PROGRAMMING

Fakultas Vokasi | D4TRPL | Institut Teknologi Del
Week 2 Sesi 1 - CDN - Tahun Ajaran 2025/2026



Konten



- Struktur Program Java
- Variabel & Tipe data
- Konstanta & Casting
- Array (1D & multidimensi)
- String (sebagai objek referensi)
- Operator (aritmatika, relasi, logika, bitwise, assignment, ternary)

- Percabangan (if, if-else, if-else-if, switch)
- Perulangan (for, while, do-while, for-each)
- Break & Continue
- Input/output (Scanner)
- Implementasi Sederhana

Struktur Program Java

- Semua kode Java ditulis dalam class.
- Eksekusi selalu dimulai dari method main().
- Standar penggunaan (Oracle, 2023; Deitel & Deitel, 2018):
 - Nama file = nama class public (HelloWorld.java).
 - Nama class pakai PascalCase.
 - main() harus ditulis public static void main(String[] args).

- Contoh :

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Run Program Java

- Gunakan IntelliJ IDEA, simpan file, misalnya HelloWorld.java
- Tekan tombol 'Run', dan Ya! "Hello, World!" akan muncul pada konsol.
- Penguraian eksekusi kode program :
 - class sebagai kelas yang akan dijalankan,
 - public static void main(String[] args) sebagai pintu masuk program,
 - System.out.println sebagai cara untuk mencetak ke konsol.

- Contoh :

```
public class HelloWorld {  
    public static void main(Strin  
    ...  
        System.out.println("Hello  
    }  
}
```

Output

Hello, World!

=== Code Execution Successful ===

Variabel & Tipe Data

Keduanya untuk mengelola data secara efisien — `int` dan `boolean` untuk penanganan nilai langsung, `String` untuk **struktur kompleks**, serta `array` dan `enum` untuk penyimpanan terstruktur dan nilai konstan.

- **Variabel** merupakan tempat menyimpan data dalam memori.
- **Tipe data primitive:**
 - `byte` (8 bit), `short` (16 bit), `int` (32 bit), `long` (64 bit)
 - `float` (32 bit), `double` (64 bit)
 - `char` (16 bit, Unicode)
 - `boolean` (`true/false`)
- **Tipe data non-primitive (referensi):** `String`, `Array`, `Class`, `Interface`, etc.

Jenis Variabel

Dalam Java, istilah "field" sebagai object dan "variabel" sama-sama digunakan dan sering kali merujuk pada hal yang sama, sehingga membingungkan para pengembang baru.

Java mendefinisikan jenis-jenis variabel berikut:

1. Instance Variables (Non-Static Fields)

- Milik setiap object (unik untuk tiap instance).
- Contoh: `int speed;` → tiap objek punya nilai sendiri.

2. Class Variables (Static Fields)

- Hanya 1 salinan untuk semua objek dari class tsb.
- Dideklarasikan dengan **static**.
- Contoh: `static int jumlahMahasiswa;`

3. Local Variables

- Dideklarasikan di dalam method.
- Hanya bisa diakses dalam method itu saja.

◦ Contoh:

```
void hitung() {  
    int count = 0; // local variable  
}
```

4. Parameters

- Variabel input untuk method/constructor.

◦ Contoh:

```
void setName(String nama) {  
    this.nama = nama;  
}
```

Penamaan Variabel

Aturan dan konvensi untuk penamaan variabel dalam Java, diringkas sebagai berikut (Schildt, 2019; Oracle, 2023) :

- Case-sensitive (besar kecil huruf dari nama variabel → berbeda).
- Nama harus mulai dengan huruf (tidak boleh mulai dengan angka atau simbol).
- Hindari \$ dan _, kecuali dalam kasus tertentu.
- Tidak boleh pakai keyword Java (class, int, if, dll.).
- Gunakan nama yang jelas & mudah dipahami (totalNilai, bukan tn).
- Gunakan final untuk konstanta.
- Konvensi:
 - Variabel → camelCase (totalGaji, rataNilai).
 - Konstanta → UPPER_CASE dengan _ (MAX_SIZE, PI).

Contoh:

```
int umur = 20;
double gaji = 3500000.50;
boolean lulus = true;
String nama = "Budi";
```

Tipe Data Primitive

Java adalah statically-typed language, dimana setiap variabel harus dideklarasikan dengan tipe sebelum digunakan. Terdapat **8 tipe data primitive di Java** (Schildt, 2019; Oracle, 2023), yaitu:

- byte → 8-bit, range -128 s/d 127. Hemat memori, cocok untuk array besar.
- short → 16-bit, range -32.768 s/d 32.767. Juga untuk hemat memori.
- int → 32-bit, range -2^{31} s/d $2^{31}-1$. Tipe bilangan bulat yang paling umum.
- long → 64-bit, range -2^{63} s/d $2^{63}-1$. Untuk bilangan sangat besar.
- float → 32-bit, single precision angka pecahan. Tidak cocok untuk keuangan (gunakan BigDecimal).
- double → 64-bit, double precision angka pecahan. Pilihan utama untuk bilangan desimal.
- boolean → hanya true atau false. Dipakai untuk logika & kondisi.
- char → 16-bit Unicode, menyimpan 1 karakter. Range: \u0000–\uFFFF (65.535 karakter).
- Tambahan :
 - String bukan tipe primitive, tapi tipe objek khusus untuk teks.
 - String bersifat immutable (tidak bisa diubah setelah dibuat).

Tipe Data Non-Primitive

Disebut juga reference types, yang menyimpan referensi (alamat memori), bukan nilai langsung. Umumnya user-defined, kecuali beberapa yang disediakan Java (String, Array) dan menjadi dasar dari Object-Oriented Programming (OOP) (Schildt, 2019; Oracle, 2023). Beberapa Non-Primitive tipe data:

- **Class:** cetak biru (blueprint) untuk membuat objek → berisi atribut (field) & perilaku (method).

```
class Demo {  
    int a, b;  
    Demo(int a, int b){ this.a=a; this.b=b; }  
    int addition(){ return a+b; }  
}
```

- **String:** objek khusus untuk teks (bukan primitive).

```
String s1 = "Hello";  
String s2 = new String("Java");
```

Tipe Data Non-Primitive

Disebut juga reference types, yang menyimpan referensi (alamat memori), bukan nilai langsung. Umumnya user-defined, kecuali beberapa yang disediakan Java (String, Array) dan menjadi dasar dari Object-Oriented Programming (OOP) (Schildt, 2019; Oracle, 2023). Beberapa Non-Primitive tipe data:

- **Array** : koleksi elemen bertipe sama, diakses via indeks.

```
int[] arr = {1, 2, 3};  
double[] arr2 = {1.1, 2.2};
```

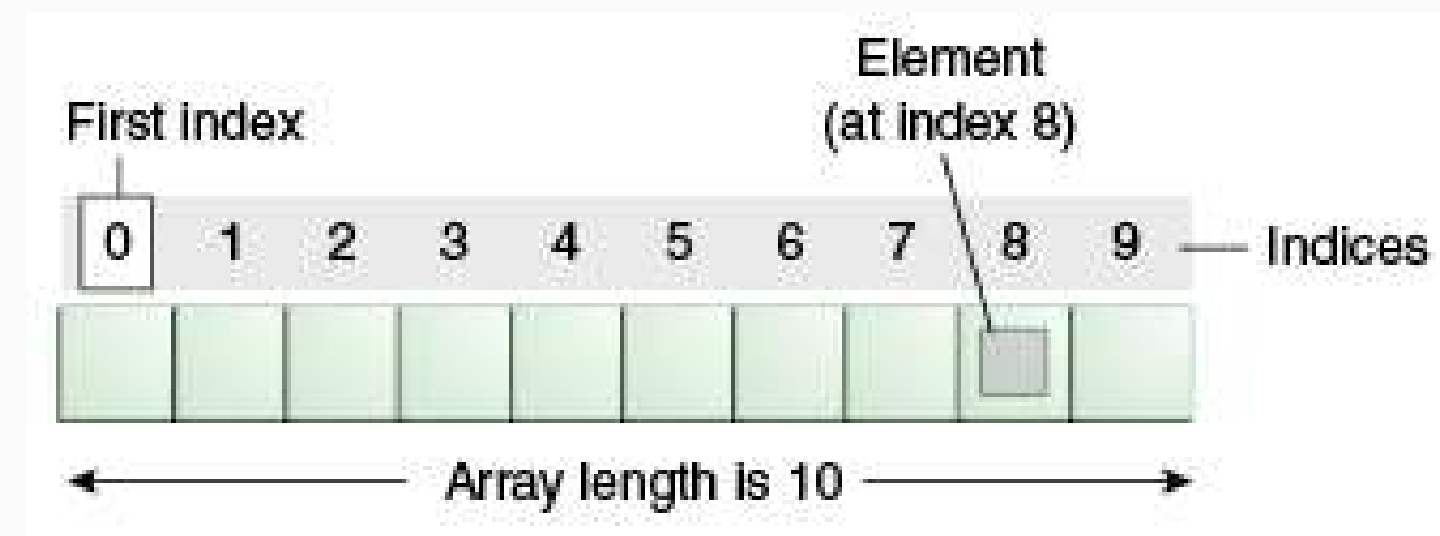
- **Interface**: Kontrak berisi method tanpa implementasi. Merupakan class yang meng-implement wajib isi semua method.

```
interface Operasi {  
    int tambah(int a,int b);  
}  
class Kalkulator implements Operasi {  
    public int tambah(int a,int b){ return a+b; }  
}
```

Array dalam Java

Array diibaratkan wadah (container) untuk menyimpan sekumpulan nilai bertipe sama.

- Ukuran array ditentukan saat dibuat dan tidak bisa diubah.
- Elemen array diakses dengan indeks yang dimulai dari 0.



Deklarasi Array

```
// deklarasi array integerint[] anArray;
```

- Bentuk umum: tipe[] namaArray;
- Bracket [] sebaiknya ditulis setelah tipe data (standar Java).

Membuat & Inisialisasi Array

```
// buat array 10 elemen  
anArray = new int[10];
```

```
// isi nilai  
anArray[0] = 100;  
anArray[1] = 200;
```

Array dalam Java

Shortcut Inisialisasi Array

```
int[] anArray = {100,200,300,400};
```

Akses Elemen Array

- Gunakan loop (for/for-each) untuk mengakses semua elemen.
- Gunakan .length untuk mengetahui ukuran array.

```
System.out.println(anArray[0]); // 100
System.out.println(anArray[1]); // 200
```

Array Multidimensi

- Array di dalam array (String[][]).
- Setiap baris bisa memiliki panjang berbeda.

```
String[][] names = {
    {"Mr. ", "Mrs. ", "Ms. "},
    {"Smith", "Jones"}
};
System.out.println(names[0][0] + names[1][0]); // Mr. Smith
System.out.println(names[0][2] + names[1][1]); // Ms. Jones
```

Catatan Penting

- Array di Java berbeda dari C/Fortran karena panjang baris bisa bervariasi.
- Jika array belum diinisialisasi, akan muncul error saat diakses.
- Gunakan ArrayList jika butuh ukuran dinamis.

String dalam Java

String merupakan komponen fundamental dalam Java. String menjadi objek yang memfasilitasi pembuatan dan manipulasi serangkaian karakter (Schildt, 2019).

- String adalah objek, bukan primitive.
- Bisa dibandingkan dengan `.equals()` (bukan `==`).
 - Gunakan `equals()` untuk cek kesamaan isi string.
 - `==` hanya cek referensi (alamat memori).
 - **Contoh:**

```
String nama = "Budi";  
System.out.println(nama.length());    // 4  
System.out.println(nama.toUpperCase()); // "BUDI"  
System.out.println(nama.equals("Budi")); // true
```

Konversi Tipe Data

Type casting adalah menerjemahkan satu tipe data ke tipe data lain, baik secara eksplisit maupun implisit (Schildt, 2019). Apa itu? Jadi, ada 2 jenis utama:

1. **Implicit** (automatic/ widening), bersifat aman, tidak ada kehilangan data.

- Konversi dari tipe kecil → tipe besar (contoh int → double).
- Dilakukan otomatis oleh compiler.

```
int myInt = 9;  
double myDouble = myInt; // otomatis
```

2. **Explicit** (manual/ narrowing), berisiko kehilangan data, butuh deklarasi khusus.

- Konversi dari tipe besar → tipe kecil (contoh double → int).
- Harus ditulis manual dengan tanda (tipe).

```
double myDouble = 9.78;  
int myInt = (int) myDouble; // hasil 9, .78 hilang
```

Operator yang didukung oleh bahasa pemrograman Java, diantaranya:

- **Assignment, Arithmetic, and Unary Operators**
- **Equality, Relational, and Conditional Operators**
- **Bitwise and Bit Shift Operators**

Operator dalam Java

Operators merupakan simbol untuk melakukan operasi pada nilai/variabel (dipakai sangat sering dalam logika program). Operator yang didukung oleh bahasa pemrograman Java:

Simple Assignment Operator

= Simple assignment operator

Arithmetic Operators

+	Additive operator (also used for String concatenation)
-	Subtraction operator
*	Multiplication operator
/	Division operator
%	Remainder operator

Contoh:

```
// Contoh penggunaan operator
int sum = 3 + 4;           // 7
int diff = 10 - 3;         // 7
int prod = 7 * 3;          // 21
double q = 20.0 / 3;       // 6.666...
int r = 7 % 3;             // 1
```

Operator dalam Java

Operators merupakan simbol untuk melakukan operasi pada nilai/variabel (dipakai sangat sering dalam logika program). Operator yang didukung oleh bahasa pemrograman Java:

Unary Operators

- `+` Unary plus operator; indicates positive value (numbers are positive without this, however)
- `-` Unary minus operator; negates an expression
- `++` Increment operator; increments a value by 1
- `--` Decrement operator; decrements a value by 1
- `!` Logical complement operator; inverts the value of a boolean

Contoh:

```
// Compound assignment: +=, -=, *=, /=, %=  
int x = 10;  
x += 5; // x = 15  
  
//Dampak tipe data:  
double d = 10.0/3; // 3.333...  
int i = 10/3;      // 3 (truncation)  
  
//Pada ekspresi campuran, tipe lebih kecil dipromosikan  
//(mis. int → double). Math utility:  
double p = Math.pow(2, 3); // 8.0  
double s = Math.sqrt(64);  // 8.0
```

Operator dalam Java

Operators merupakan simbol untuk melakukan operasi pada nilai/variabel (dipakai sangat sering dalam logika program). Operator yang didukung oleh bahasa pemrograman Java:

Equality and Relational Operators

<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code>>=</code>	Greater than or equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to

Conditional Operators

<code>&&</code>	Conditional-AND
<code> </code>	Conditional-OR
<code>?:</code>	Ternary (shorthand for if-then-else statement)

Type Comparison Operator

<code>instanceof</code>	Compares an object to a specified type
-------------------------	--

Contoh:

```
//Operator == != > < >= <= → hasil boolean.
```

```
int a = 5;
```

```
boolean ok = (a == 5); // true
```

```
//Operator: && (AND), || (OR), ! (NOT)
```

```
boolean r1 = (5 > 3) && (7 < 10); // true
```

```
boolean r2 = (5 < 3) || (7 < 10); // true
```

```
boolean r3 = !(5 > 3); // false
```

Operator dalam Java

Operators merupakan simbol untuk melakukan operasi pada nilai/variabel (dipakai sangat sering dalam logika program). Operator yang didukung oleh bahasa pemrograman Java:

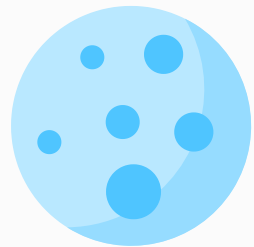
Bitwise and Bit Shift Operators

~	Unary bitwise complement
<<	Signed left shift
>>	Signed right shift
>>>	Unsigned right shift
&	Bitwise AND
^	Bitwise exclusive OR
	Bitwise inclusive OR

Contoh:

```
//Operator: Bitwise (untuk integer): & | ^ ~ << >>  
  
int and = 5 & 3; // 1  
int or  = 5 | 3; // 7  
int xor = 5 ^ 3; // 6
```

Java : Expression, Statement, dan Blocks



1. Expressions

- Kombinasi variabel, operator, dan pemanggilan method untuk menghasilkan 1 nilai.
 - Bisa berupa compound expression (gabungan), misalnya $(x + y) / 100$.
 - Gunakan tanda kurung () untuk memperjelas urutan eksekusi → kode lebih mudah dibaca & aman dari ambiguitas.
 - Contoh :

```
int result = 1 + 2;           // menghasilkan int
if (a == b) System.out.println("Sama"); // menghasilkan boolean
```

Java : Expression, Statement, dan Blocks

2. Statement

- Unit lengkap eksekusi program (ibarat "kalimat" dalam bahasa alami). Jenis utama:

- **Expression statement** : hasil ekspresi yang diakhiri ;

```
aValue = 8933.234;    // assignment
aValue++;             // increment
System.out.println(a); // method call
new Bicycle();         // object creation
```

- **Declaration statement** : deklarasi variabel

```
double aValue = 8933.234;
```

- **Control flow statement** : mengatur alur eksekusi (akan dibahas selanjutnya).



Java : Expression, Statement, dan Blocks

3. Blocks

- Sekumpulan statement diapit { ... }. Digunakan di dalam struktur kontrol (if-else, loop, dsb).

◦ Contoh :

```
if (condition) {  
    System.out.println("True");  
} else {  
    System.out.println("False");  
}
```

Hint: Control statement nantinya akan menggabungkan **ekspresi**, dijalankan sebagai **statement**, lalu dikelompokkan dalam **block** untuk membentuk logika program yang lebih kompleks. **Expression** → **Statement** → **Block** adalah dasar sebelum control statement.

Statement control menawarkan cara dinamis untuk mengendalikan alur program.

- **The if-then and if-then-else Statements**
- **The switch Statement**
- **The while and do-while Statements**
- **The for Statement**
- **Branching Statements**

Control Statements

1. If–Then & If–Then–Else Statement

Menjalankan blok kode bila kondisi true; sediakan jalur alternatif dengan else.

- Tetap gunakan { } meski 1 baris → mencegah bug saat menambah statement baru.
- Susun kondisi dari paling spesifik → umum (kaskade else if untuk rentang nilai).

```
if (condition) {  
    // Block of code to be executed if the condition is true  
}
```

```
int age = 20;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

```
if (condition) {  
    // Block of code executed if condition is true  
} else {  
    // Block of code executed if condition is false  
}
```

```
int age = 15;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
} else {  
    System.out.println("You are not eligible to vote.");  
}
```

```
int marks = 75;  
if (marks >= 85) {  
    System.out.println("Grade A");  
} else if (marks >= 70) {  
    System.out.println("Grade B");  
} else {  
    System.out.println("Grade C");  
}
```



Control Statements

2. The switch Statement

Banyak cabang berdasarkan satu nilai (int, char, enum, String, wrapper).

- Tiap case umumnya diakhiri break → hindari fall-through tak sengaja.

```
int month = 8; String name;
switch (month) {
    case 1: name="January"; break;
    // ...
    case 8: name="August"; break;
    default: name="Invalid"; break;
}
```

- Bisa multi-label, contoh:

```
switch (month) {
    case 1: case 3: case 5: case 7: case 8: case 10: case 12: days=31; break;
    case 4: case 6: case 9: case 11: days=30; break;
    case 2: days = isLeap(year)?29:28; break;
    default: /* handle invalid */
}
```

Control Statements

3. The while and do-while Statements

Mengulang blok kode beberapa kali, didorong oleh kondisi tertentu.

- While: cek di awal; bisa 0 kali eksekusi.

```
int i=1;
while (i<=10) { System.out.println(i); i++; }
```

- Do-while: cek di akhir; pasti eksekusi ≥ 1 kali.

```
int j=1;
do { System.out.println(j); j++; } while (j<=10);
```

Tips: while(true) { ... } = loop tak hingga (jadi, butuh break di dalamnya).

Control Statements

4. The for Statement

Melakukan iterasi pada rentang nilai atau elemen dalam suatu koleksi.

- Perulangan **for** ini biasanya digunakan ketika jumlah iterasi telah diketahui sebelumnya.

```
for (int k=1; k<=10; k++) {  
    System.out.println(k);  
}
```

- Keterangan:

- Variabel kontrol (k) berlaku di dalam blok for (membatasi cakupan → lebih aman).
- Semua ekspresi boleh kosong → `for(;;){}` adalah loop tak hingga.

- Enhanced for (lebih ringkas untuk array/koleksi), bila tidak butuh indeks:

```
int[] nums = {1,2,3,4,5};  
for (int n : nums) {  
    System.out.println(n);  
}
```

Control Statements

5. Branching Statements

Pernyataan yang berguna untuk mengubah alur eksekusi normal di dalam loop atau method.

- `break` (unlabeled): keluar atau menghentikan dari loop/switch terdekat lebih awal.

```
int[] a={32,87,3,589,12};  
int target=12; int idx=-1;  
for (int i=0;i<a.length;i++){  
    if(a[i]==target){ idx=i; break; }  
}
```

- `break` berlabel: keluar keluar atau menghentikan dari loop/switch dari outer loop tertentu.

```
search:  
for (int i=0;i<m.length;i++){  
    for (int j=0;j<m[i].length;j++){  
        if(m[i][j]==target){ found=true; break search; }  
    }  
}
```

Control Statements

5. Branching Statements

Pernyataan yang berguna untuk mengubah alur eksekusi normal di dalam loop atau method.

- continue: melewati iterasi saat ini, lanjut ke iterasi berikutnya.

```
int count=0; String s="peter piper picked...";
for (int i=0;i<s.length();i++){
    if (s.charAt(i)!='p') continue;
    count++;
}
```

- return: hentikan eksekusi method saat ini, lalu kembali ke pemanggil method (opsional dengan nilai).

```
if (x<0) return;
return ++count; // harus cocok dengan tipe hasil method
```

Input/Output (Scanner)

Scanner adalah pelengkap Formatter pada Java. Scanner membaca input yang diformat dan mengubahnya menjadi bentuk biner.

- Secara umum, Scanner membaca token dari sumber yang mendasarinya yang Anda tentukan saat Scanner dibuat
- Scanner mencakup banyak pola yang telah ditentukan sebelumnya, yang cocok dengan tipe primitif, seperti int dan double, serta string.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Masukkan nama: ");
String nama = sc.nextLine();
System.out.println("Halo, " + nama);
```

Simple Implementation

Menghitung nilai rata-rata mahasiswa, dengan spesifikasi:

- Gunakan enhanced for loop (for-each) untuk array agar lebih ringkas.
- Casting ke double untuk menghindari pembagian integer.

```
int[] nilai = {80, 90, 75, 88};  
int total = 0;  
for(int n : nilai){ // enhanced for loop  
    total += n;  
}  
double rata = (double)total/nilai.length;  
System.out.println("Rata-rata = " + rata);
```

Referensi

- Deitel, P. J., & Deitel, H. M. (2018). Java How to Program (11th Edition). Pearson.
- Schildt, H. (2019). Java: The Complete Reference (11th Edition). McGraw-Hill.
- Oracle. (2023). The Java™ Tutorials. Oracle Docs
- Aslanyan, V. (2023, November 9). Learn Java Fundamentals – Object-Oriented Programming [Full Book].
 - freeCodeCamp.org. <https://www.freecodecamp.org/news/learn-java-object-oriented-programming/#heading-understanding-java-syntax>

TERIMA KASIH

Stay Happy & Healthy